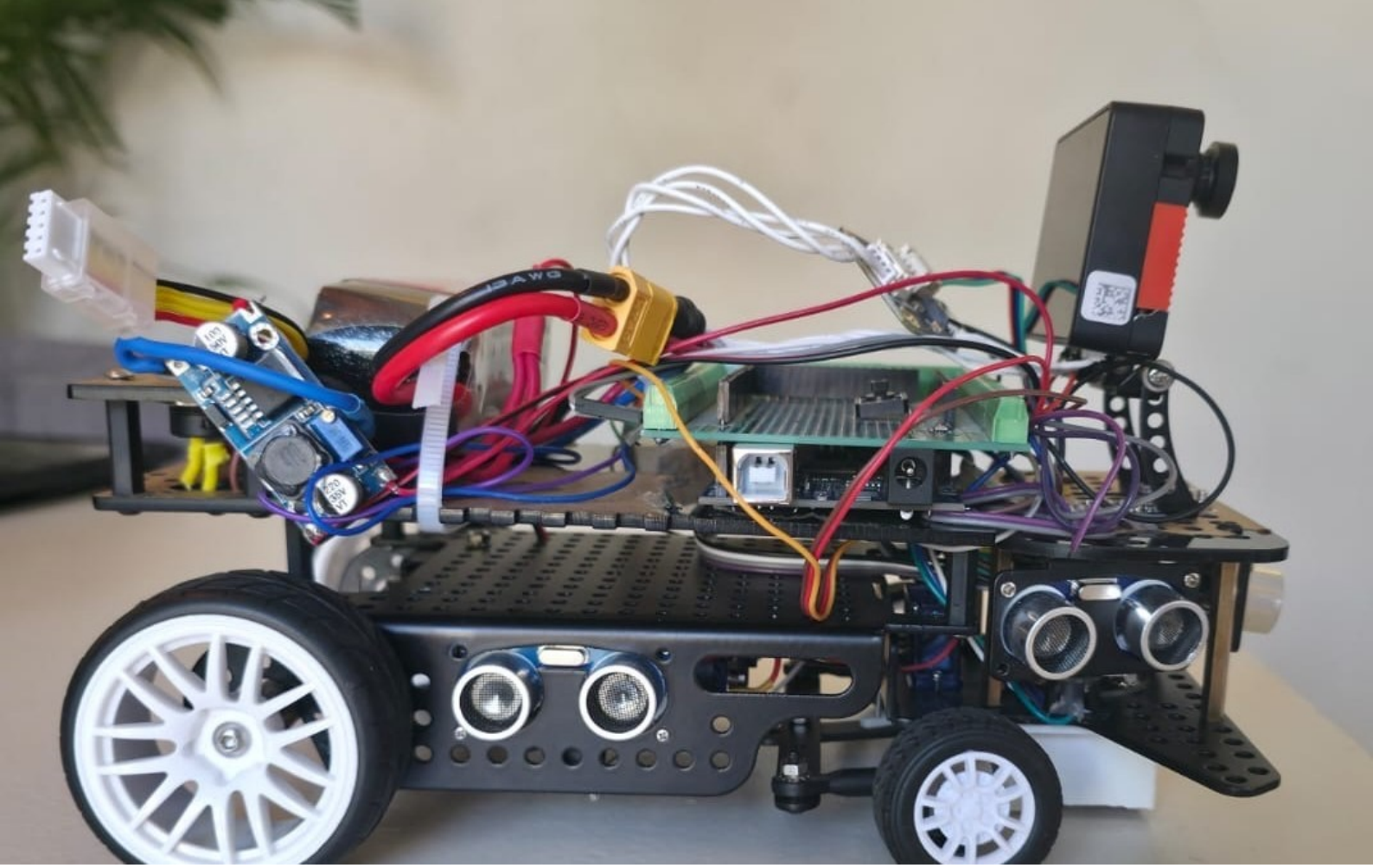


The Invader

Engineering notebook · WRO 2026 Future Engineers · National Final

---

Team	KFS Engineers, King Faisal Schools, Riyadh
Members	Mohammed Albassam, Layth Alzahrani
Coach	Eng. Mohamed Salaheldin Abdulhaq

A self-driving car built on an Arduino Mega, with rear-wheel drive, servo steering and a HuskyLens 2 camera for the obstacle round.

Contents

We organized this notebook around the five criteria in the WRO 2026 documentation rubric (Appendix C of the rules), so each part can be judged on its own.

	Section	Page
	The team, our journey, the Invader at a glance	3
1	Mobility and mechanical design	6
2	Power and sensors	11
3	Software and obstacle strategy	16
4	Engineering decisions: tradeoffs, risks, testing	21
5	Reproducibility: build, run, versions	24
	Photo record and sources	26
A	Open Challenge code (current version)	28
B	Obstacle Challenge code (qualifier version)	32

Rubric map

Rubric criterion (6 pts each)	Where to look
1 Mobility and mechanical design	Drive and torque reasoning p.7, steering p.8, size limits p.9, iterations p.10
2 Power and sensor architecture	Power budget p.12, sensor placement p.14, calibration p.15
3 Software and obstacle strategy	State machine p.16, edge cases p.17, HuskyLens plan p.20
4 Systems thinking and decisions	Tradeoffs p.21, risks p.22, test results p.23
5 Reproducibility and GitHub	Build guide p.24, versions p.25, code p.28

The team



Mohammed Albassam

Hardware and documentation

Built the chassis and drive, wired the electronics and keeps this notebook. He comes from VEX robotics, where he competed at VEX Worlds in Dallas.



Layth Alzahrani

Programming and electronics

Writes the code that runs the car, and handles the electronic connections and sensor integration.



Eng. Mohamed Salaheldin Abdulhaq

Coach

Has coached robotics teams in Saudi Arabia since 2010. Gives us design reviews and helps us troubleshoot.

Why “the Invader”?

Our coach started coaching robotics in Saudi Arabia in 2010. Since then, every robot he has worked with has carried the same name: WRO, VEX, WorldSkills, sumo, all of them were called the Invader. Sixteen years later the tradition is still going, so of course our car is the Invader too.

We like that the name comes with history, and we want this Invader to live up to it.

How we split the work. Mohammed owns the hardware and the notebook, Layth owns the code.

Our journey

Here is how the Invader got to where it is now: what we built, what broke, and what we learned along the way.

Starting point

We built the car on the RN AI kit chassis: rear drive with a differential, servo steering at the front. On top went an Arduino Mega, an L298N motor driver, a TCS3200 color sensor facing the floor, five ultrasonic sensors and a HuskyLens 1, all running off a 12 V Studica battery.

First test, first failure

In our very first test run an ultrasonic sensor wire hung into a wheel, and the wheel **cut straight through it**. While we were replacing it, the Arduino power cord broke too, and of course we had no spare. We soldered it, covered it in heat-shrink and zip-tied every cable away from the wheels. Since then, checking cable clearance is part of every build and every run.

Three clean laps

The first Open Challenge program turned left on every blue line. Our first two logged runs went into tuning the steering. Compared with our earlier sketch, straight-ahead moved from 80° to 82° and the straight speed dropped from PWM 170 to 140. After that, **three runs in a row finished all three laps** and stopped in the right place.

The qualifier

We didn't have time to get the camera working, so for the Obstacle Challenge we drove a fixed list of timed moves tuned on our practice layout. It was simple, but it worked well enough to **qualify us for the national final**.

Version 2

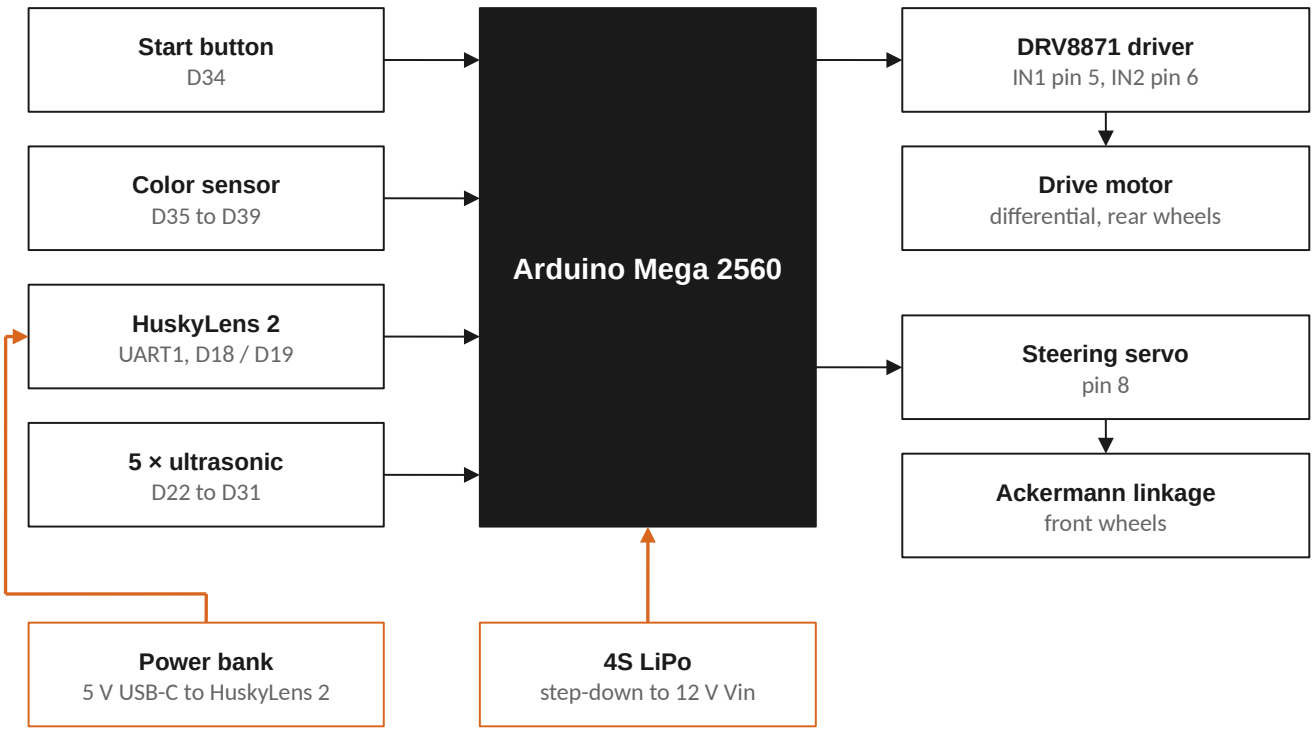
For the final we swapped the L298N for a DRV8871 driver and rewrote the Open Challenge code. The car now **picks its direction from the first line it sees** (blue means anticlockwise, orange means clockwise), so it can handle either direction the judges draw. The sensor loop also got about 19 times faster in the worst case (page 18). We also swapped the Studica battery for a **4S LiPo**, with a step-down regulator holding the Mega's Vin at 12 V, and added the ultrasonic sensors to the newest version of the Open Challenge code.

Giving it eyes

We moved to a HuskyLens 2 for the obstacles. Plugged into the Arduino, **it wouldn't even turn on**. The Mega's 5 V pin simply can't supply the current the camera needs (page 12), so the camera now runs off its own power bank.

The Invader at a glance

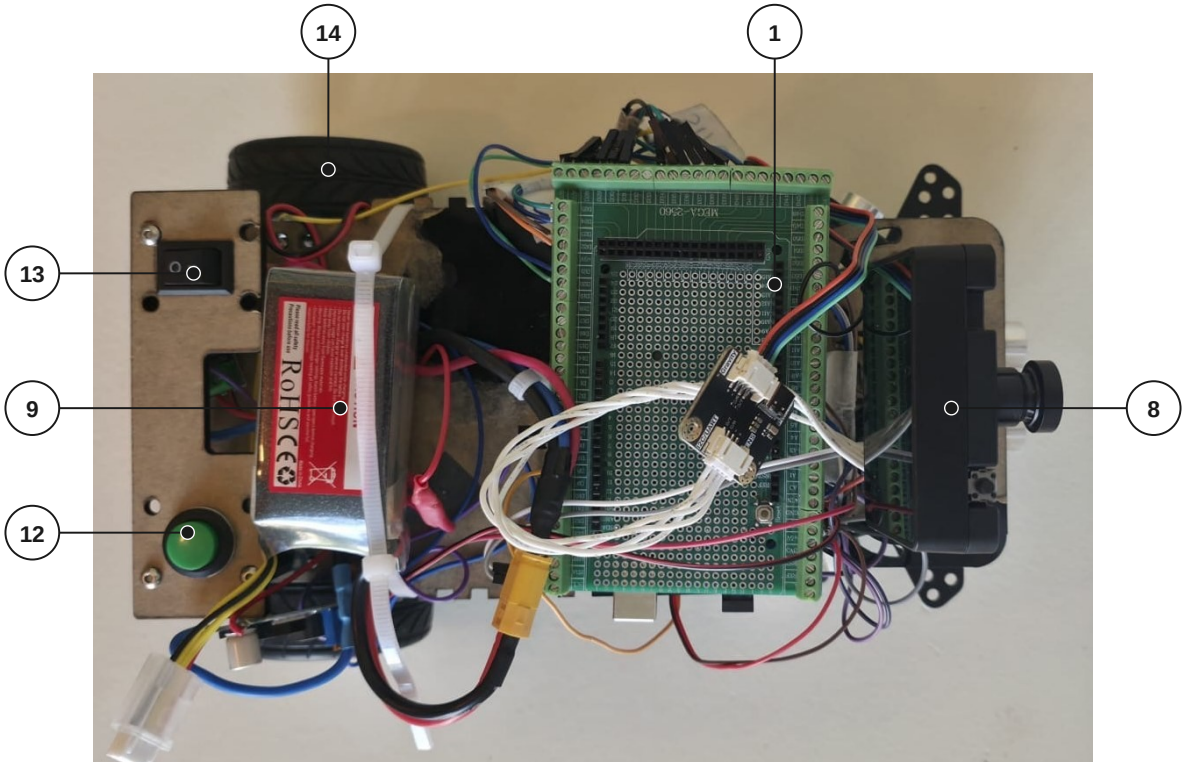
Controller	Arduino Mega 2560 with a screw-terminal shield
Drive	One DC gear motor, rear differential, DRV8871 driver
Steering	Servo on pin 8, Ackermann linkage on the front wheels
Line sensing	TCS3200 color sensor, 27.5 mm above the floor
Vision	HuskyLens 2 on UART, powered by its own power bank
Distance	5 × HC-SR04 ultrasonic, read by the newest Open Challenge code
Power	4S LiPo 14.8 V, 1500 mAh, stepped down to 12 V for the Mega + USB power bank for the camera
Size	275 × 195 × 210 mm (limit 300 × 200 × 300 mm)
Code	Arduino C++, one sketch per challenge



How the parts connect. Black arrows are signals, orange arrows are power.

1.1 Layout and parts list

The Invader has two decks. The lower deck is the RN AI kit's metal chassis with the steering and the rear drive. The upper deck carries the Mega, the motor driver, the start button and the power switch, so **every plug we need is on top** and we can reach it without taking the car apart.



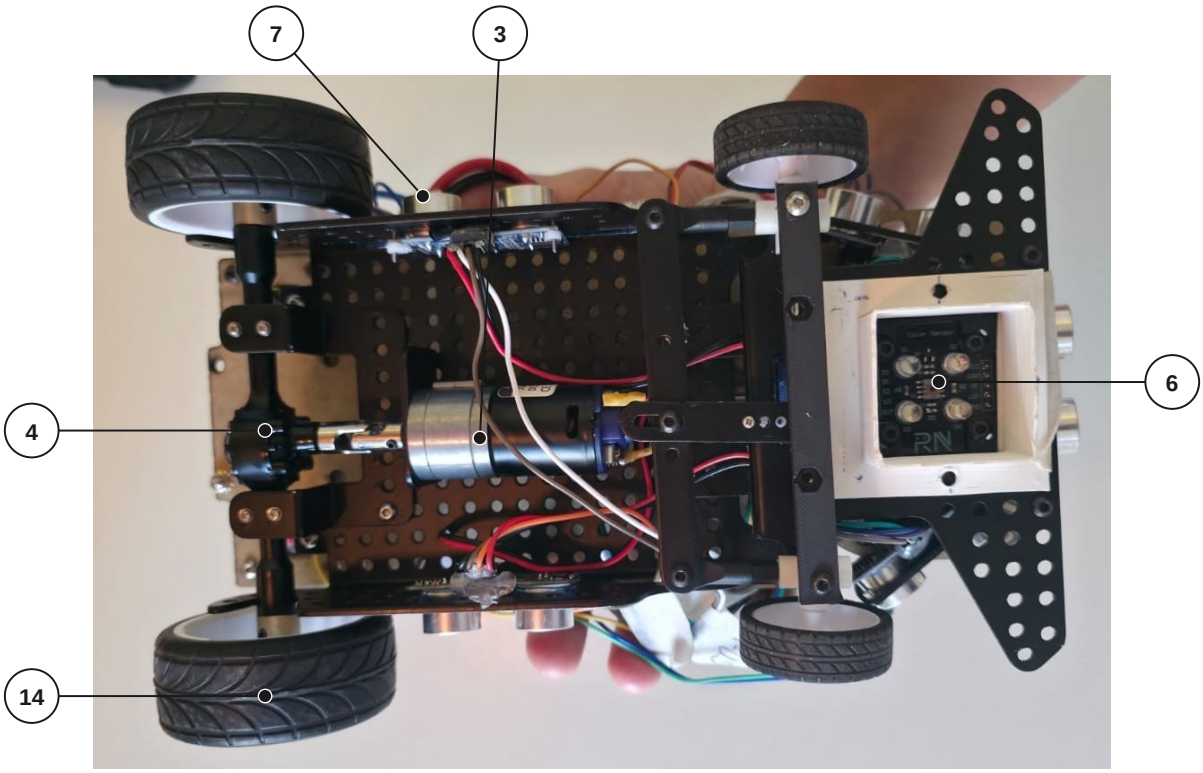
Top view, front on the right: 1 Mega on its screw shield, 8 HuskyLens 2, 9 LiPo, 12 start button, 13 power switch, 14 rear wheel.

#	Part	Qty	Job on the car
1	Arduino Mega 2560 + screw-terminal shield	1	Runs everything
2	DRV8871 motor driver	1	Speed and direction of the drive motor
3	DC gear motor	1	Drives the rear axle
4	Rear differential	1	Lets the rear wheels turn at different speeds
5	Steering servo	1	Moves the front steering linkage
6	TCS3200 color sensor (4 LEDs)	1	Reads blue and orange lines on the floor
7	HC-SR04 ultrasonic sensor	5	Distance to walls and objects
8	HuskyLens 2	1	Sees the red and green obstacles
9	4S LiPo, 14.8 V 1500 mAh (Zeee, XT60)	1	Main power
10	Step-down regulator, set to 12 V	1	Feeds the Mega's Vin
11	USB power bank	1	Powers the HuskyLens 2 only
12	Start button	1	D34, starts the run
13	Power switch	1	Main power on and off
14	Wheels	4	2 × 60 mm rear, 2 × 35 mm front

1.2 Drive: motor, differential and driver

One DC gear motor drives the rear axle through a differential. The front wheels only steer, like a real car.

Why a differential. In a corner the outside rear wheel travels further than the inside one. With a solid axle one wheel would have to skid, which wastes power and makes turns less repeatable. The differential lets the outside wheel spin faster, so the car turns cleanly.



Underside, front on the right: 3 motor, 4 differential, 6 color sensor in its white 3D-printed holder, 7 ultrasonic, 14 rear wheel.

How we drive the motor

The DRV8871 has two inputs. We hold `IN2` HIGH and send PWM to `IN1` as `255 - speed`. Between pulses the driver brakes the motor instead of letting it coast (slow decay), which gives a more even speed than letting it coast. So `NORMAL_SPEED = 200` means the motor is on about 78% of the time.

Kick start. From standstill the motor needs extra torque to get the car rolling. We give it full power (255) for 150 ms, then drop to 200.

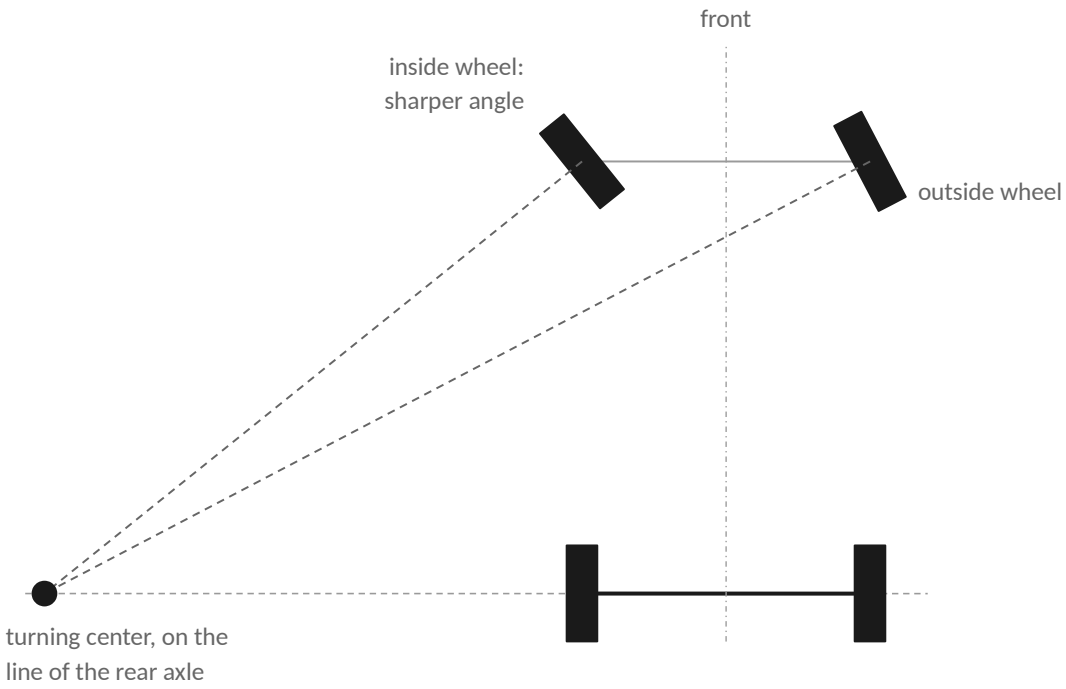
Speed. A 60 mm wheel rolls π × 60 ≈ 188.5 mm per turn, so `v = π × D × n / 60`. The kit lists a 300 rpm motor. If the wheels actually reached 300 rpm that would be about 0.94 m/s, but gearing, load and PWM all bring it down. We haven't timed the real speed yet.

1.3 Steering

One servo moves both front wheels through an Ackermann linkage. In a turn the inside wheel follows a tighter circle than the outside wheel, so the linkage turns it to a sharper angle. That way both front wheels roll instead of scrubbing sideways.

Servo angles, not wheel angles. The numbers in our code are what we send to the servo. Our straight-ahead is 82°, not the servo's 90° middle. We found 82° on the track. Our earlier sketch used 80°, and the first two logged test runs went into tuning it.

Setting	Servo	Status
`STEER_CENTER`	`82°`	Tuned on the track
`STEER_LEFT (anticlockwise corners)`	`55°`	Tested in full runs
`STEER_RIGHT (clockwise corners)`	`109°`	Mirrored estimate (82 + 27), still needs testing



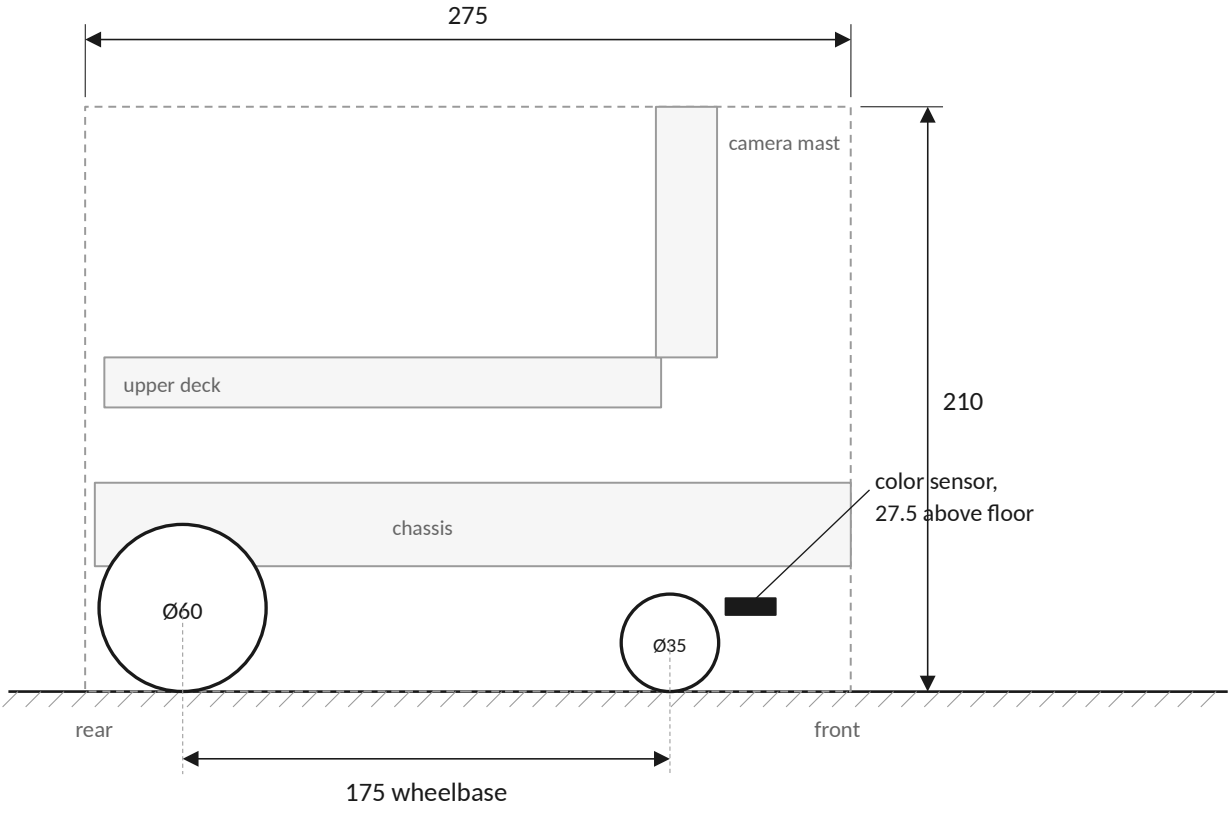
Ackermann steering in a left turn, seen from above, not to scale. Lines through both front wheels meet at one point on the rear axle line.

1.4 Size, weight and the rules

	The Invader	Limit	Spare
Length	275 mm	300 mm	25 mm
Width	195 mm	200 mm	5 mm
Height	210 mm	300 mm	90 mm
Weight	not weighed yet	1.5 kg	
Wheelbase	175 mm		
Wheels (rear / front)	60 mm / 35 mm		
Color sensor above floor	27.5 mm		

Limits from rules 11.1 and 11.2 of the WRO 2026 Future Engineers rules. Width includes everything that sticks out.

Width is the tight one. With only 5 mm to spare, anything new on the sides (the power bank, a bumper, a bracket) has to be checked against 200 mm before we mount it. We still need to weigh the car with the power bank on board.



Side view, simplified. Dimensions in mm. The dashed box is the outer envelope we measured.

1.5 Iterations and next steps

Every change below came from something that went wrong or held us back in testing.

What went wrong	What we changed	Result
An ultrasonic sensor wire hung into a wheel and got cut on the first test	Re-routed and zip-tied every cable away from the wheels, steering and drive shaft	Clearance check added to our pre-run routine
The Arduino power cord broke while we repaired the wire	Soldered it and sealed the joint with heat-shrink	Repaired on the spot, no spare needed
First two logged runs: steering and speed not tuned yet	Straight-ahead 80° to 82°, straight PWM 170 to 140 (version 1)	Runs 3, 4 and 5 all finished three laps
Version 1 could only drive anticlockwise	New code picks the direction from the first line seen	Ready for either direction
HuskyLens 2 wouldn't power on from the Mega	Gave it its own USB power bank	Camera has enough power, data still goes to the Mega

What we'd change next

Time the real speed. Run the car over 1 m and time it at a few PWM values, so speed stops being a guess.

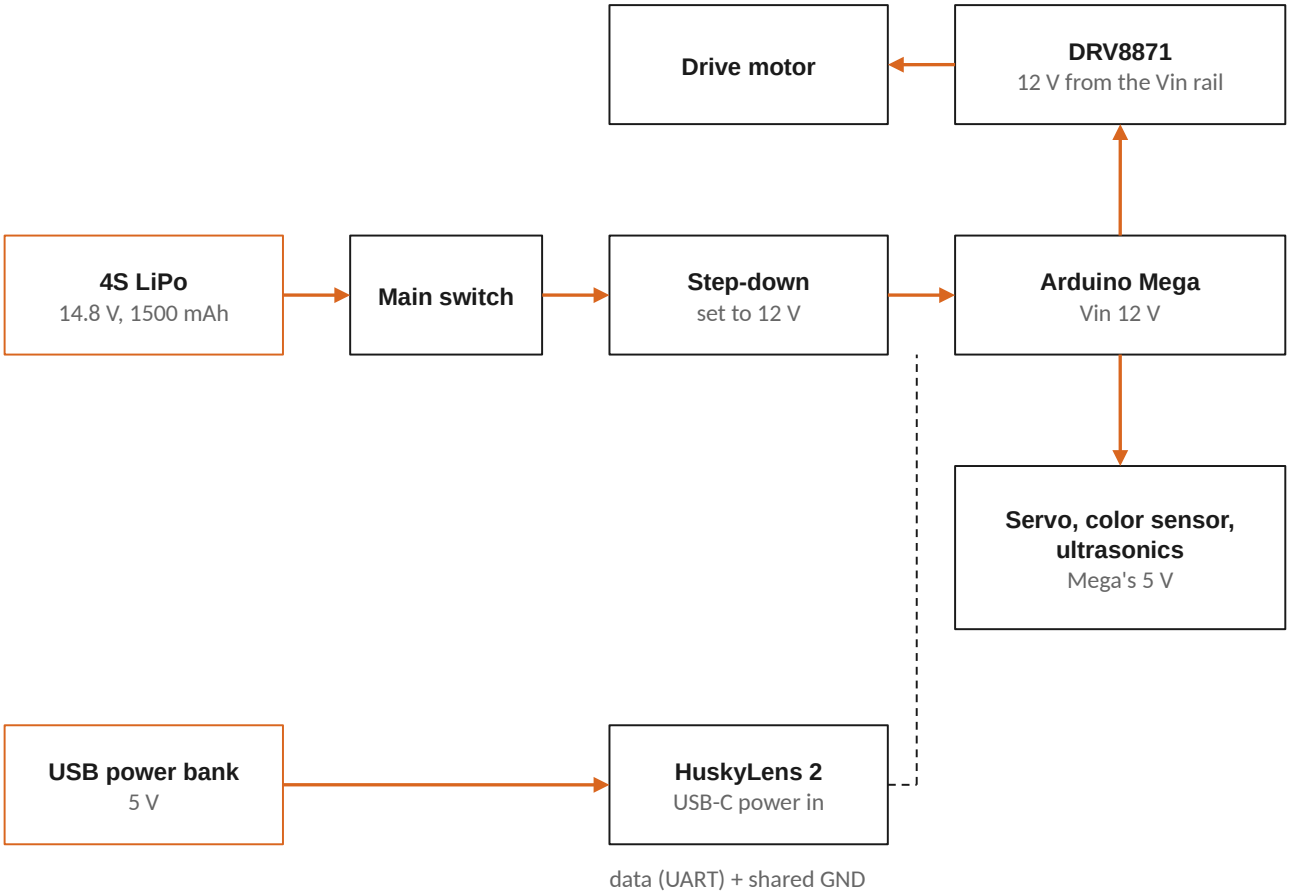
Wheel encoder. Right now every turn is a fixed time. An encoder would let us turn by distance, which wouldn't change as the battery runs down.

Mount for the power bank. It needs a proper holder inside the 200 mm width, low and near the middle to keep the center of gravity down.

Battery mount. The LiPo is held on the upper deck with zip ties. A proper strap or tray would stop it shifting if the car hits a wall.

2.1 Power architecture

A **4S LiPo** runs the car. A step-down regulator brings it down to **12 V for the Mega's Vin**, and from there the Mega feeds everything except the camera: the DRV8871 takes 12 V from the Vin rail, and the servo and sensors take 5 V from the Mega's own regulator. A USB power bank runs only the HuskyLens 2, sharing a ground wire with the Mega so they can still talk.



Power paths. Orange is power, the dashed line is the camera's data and ground wire to the Mega.

Part	Rating	What it means for us
4S LiPo (Zeee)	14.8 V nominal, 16.8 V full, 1500 mAh, 22.2 Wh, XT60	Don't drain below about 3.3 V per cell (13.2 V). Charge only with a balance charger
Step-down regulator	set to 12 V out	Holds 12 V while the LiPo drops from 16.8 V, so the motor and Mega see the same voltage all run
Mega Vin	7 to 12 V recommended	12 V sits at the top of the range
DRV8871 driver	6.5 to 45 V, 3.6 A peak	12 V is well inside its range
HuskyLens 2	3.3 to 5 V, 1.5 to 3 W	Needs more current than the Mega's 5 V pin can give (page 12)

2.2 Power budget

Load	Powered from	Current	Source
HuskyLens 2	Power bank, 5 V	`0.3 to 0.6 A`	DFRobot: 1.5 to 3 W
5 × HC-SR04	Mega 5 V	`5 × 15 mA = 75 mA`	HC-SR04 datasheet
Drive motor	12 V rail via DRV8871	`up to 3.6 A peak`	DRV8871 limit (TI)
Color sensor + LEDs	Mega 5 V	`not measured`	
Steering servo	Mega 5 V	`not measured`	peaks while turning
Arduino Mega	LiPo via step-down, 12 V	`not measured`	

We haven't measured every current yet, so this table mixes datasheet values and gaps. The gaps can be filled by measuring each load with a multimeter in series. Even with gaps, the table was enough to explain our biggest power problem.

Why the HuskyLens 2 wouldn't turn on

We first plugged the camera into the Mega like any other sensor. It never turned on.

The reason is current, not voltage. The Mega's 5 V pin comes from a small regulator on the board that turns 12 V into 5 V by **burning the difference as heat**:

`heat = (12 V − 5 V) × 0.6 A = 4.2 W`

for the camera alone, on a tiny regulator with no heatsink, before counting the sensors on the same pin

A regulator that small overheats or lets its voltage drop well before that, and the camera pulls the most right at boot. That is the most likely reason it never started.

Fix: the camera now runs off its own USB power bank through its USB-C port. The 4-pin cable to the Mega keeps only TX, RX and GND. We **left the + wire disconnected** so the power bank and the Mega's 5 V are never tied together.

Is the power bank enough? A typical 5000 mAh bank holds about 18 Wh. At the camera's worst case of 3 W that is roughly 6 hours. A run is at most 3 minutes.

Watch the servo. It is on the same 5 V pin, and its current spikes every time it steers. If the Mega ever resets in the middle of a turn, this is the first thing we check.

Main battery runtime

The LiPo holds 14.8 V × 1.5 Ah = 22.2 Wh. Worst case, the motor pulls its 3.6 A peak the whole time plus a generous 0.5 A for everything else, all at 12 V: 4.1 A × 12 V ≈ 49 W. Assuming the step-down is about 90% efficient, the battery supplies about 55 W, so 22.2 / 55 ≈ 0.4 h, about 24 minutes. That is still 8 full 3-minute rounds at maximum load.

2.3 Pin map

Every signal wire on the Mega, and which program uses it. All grounds are joined, including the camera's.

Mega pin	Connected to	Open (v2)	Obstacle (next)
DRIVE AND STEERING			
D5 (PWM)	DRV8871 IN1	●	○
D6	DRV8871 IN2	●	○
D8	Steering servo signal	●	○
START			
D34	Start button to GND, INPUT_PULLUP	●	○
COLOR SENSOR			
D35 / D36	S0 / S1, output scaling (set to 20%)	●	
D37 / D38	S2 / S3, filter select	●	
D39	OUT, pulse output	●	
HUSKYLENS 2 (UART1)			
D19 (RX1)	HuskyLens TX		○
D18 (TX1)	HuskyLens RX		○
ULTRASONIC, TRIG / ECHO (newest Open Challenge version)			
D22 / D23	Left	●	
D24 / D25	Front-left	●	
D26 / D27	Front	●	
D28 / D29	Front-right	●	
D30 / D31	Right	●	

● used now ○ planned for the HuskyLens obstacle program

D34 uses the Mega's internal pull-up, so it reads HIGH until the button connects it to ground. The servo signal is now on D8 (it was D7 in version 1). The Open Challenge code prints its readings over USB serial at 115200 baud.

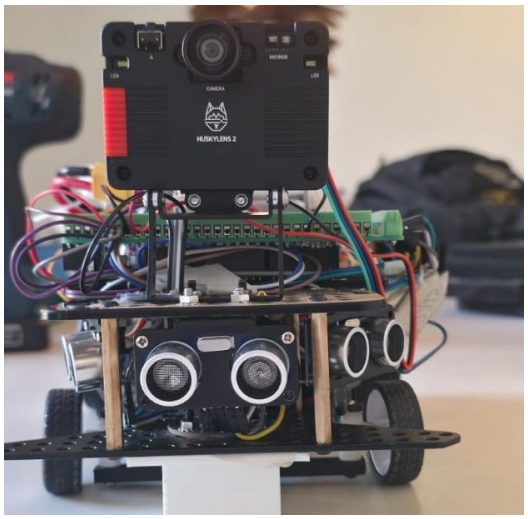
2.4 Sensors: what, where and why

Color sensor (TCS3200)

What it does: reads the blue and orange corner lines on the mat.

Where: under the front of the car in a white 3D-printed holder, 27.5 mm above the floor. At the front it reaches the corner line before the wheels do, so the turn starts on time.

Why this sensor: the lines are just colored marks on the floor, so a small sensor pointed straight down is the most direct way to read them. Its own four LEDs light the floor, so room lighting matters less.



Front view: HuskyLens 2 on the mast, the front ultrasonic sensor below it and the angled sensors at the corners.

Camera (HuskyLens 2)

What it does: finds the red and green obstacles and where they are in the image, so the car can pick a side. It sends results to the Mega over UART, so we didn't need to add a second computer.

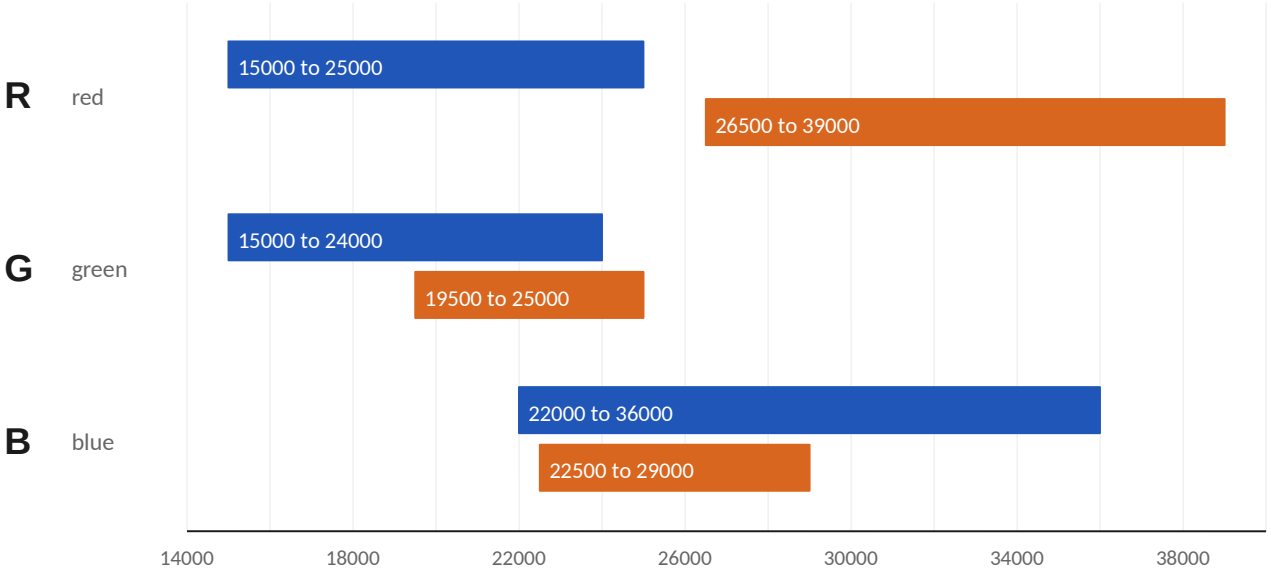
Option	What it gives us	What it costs
HuskyLens 1	Already on the car, built-in color recognition	Older, slower chip
HuskyLens 2 (our choice)	Built-in recognition, 2 MP camera, works with an Arduino over UART	Needs more power than the Mega can give, so it has its own power bank. Few FE teams have used it yet
Raspberry Pi + camera	Full control of the vision code, used by most top teams	Second computer, second power system, all vision code written from scratch

Ultrasonic sensors (5 × HC-SR04)

Left, front-left, front, front-right and right. The newest version of our Open Challenge code reads them alongside the color sensor. The code in Appendix A is the version before that, which counts lines only.

2.5 Calibrating the color sensor

Method. We fixed the sensor height at 27.5 mm and ran it with its own LEDs on. With the car connected over USB, the code prints R, G and B for every reading. We logged those readings for blue and orange and set a range for every channel around them.



Version 2 ranges for BLUE (blue bars) and ORANGE (orange bars), raw sensor units, from detectColor(). A reading must fit all three ranges.

What we learned between version 1 and 2

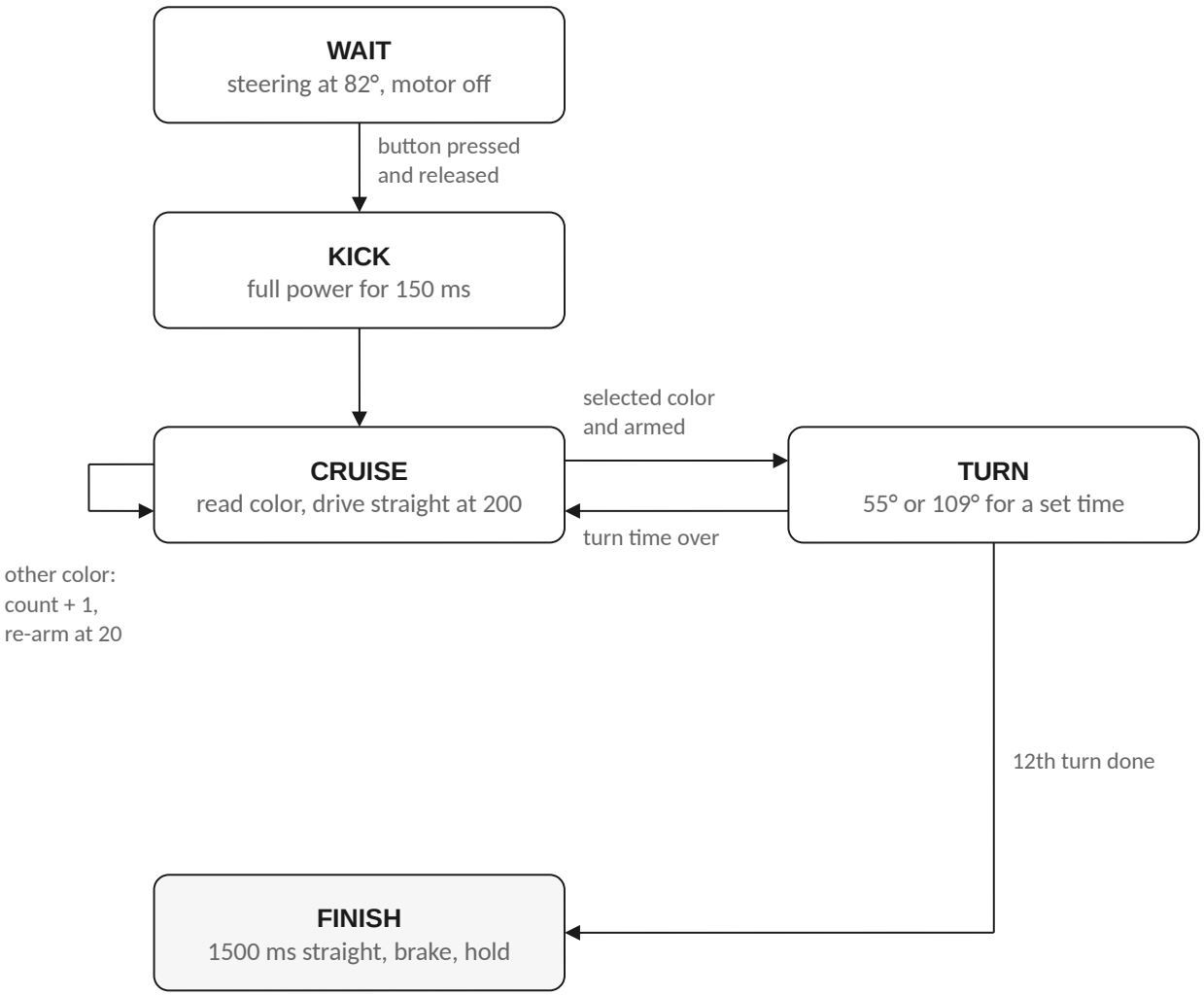
In version 1 the blue and orange ranges overlapped on **all three channels**, so we needed an extra rule (orange has clearly more red than blue) to tell them apart. After recalibrating for version 2, the **red channel alone separates them**: blue tops out at 25000 and orange starts at 26500. Green and blue still overlap, but red does the job.

Version 2 also takes one reading per channel instead of averaging three, with different timing, so its numbers are not comparable with version 1.

Weak points: the ranges belong to our mat and our lighting, and a change in sensor height shifts every number. At a new venue the readings need re-checking over serial before the first run.

3.1 Open Challenge: the state machine

Version 2 of the Open Challenge program is a small state machine. The car only ever does one thing at a time, which makes it **easy to predict and easy to debug** from the serial log.



Version 2 Open Challenge program. The first blue or orange line it meets sets the direction for the whole run.

State	What the car does	Leaves when
`WAIT`	Holds the steering at 82°, motor off	Button is pressed and released, then 300 ms pause
`KICK`	PWM 255 for 150 ms to get rolling	Always goes to CRUISE
`CRUISE`	Reads R, G, B, drives straight at 200	It reads its selected color while armed
`TURN`	Steers, counts the corner, disarms	The corner's turn time is over
`FINISH`	Drives 1500 ms, brakes 300 ms, holds	Never, until reset

3.2 Direction, corners and edge cases

Picking a direction. The judges draw the driving direction before each round (rule 9.3), so the car has to handle both. Going anticlockwise, the first line the car crosses at a corner is blue. Going clockwise, it is orange. So whichever color the car sees first sets the direction: **blue means left turns, orange means right turns**, and from then on the car only counts that color.

Counting corners. Four corners per lap, three laps, so the car finishes after **12 turns**.

Turn	Time held	Why it differs
1st	`2200 ms`	Set separately from the others
2nd to 11th	`1800 ms`	Normal corner
12th	`2100 ms`	Set separately, followed by the finish

Edge cases

What can happen	What the code does
The sensor sees the same line several times in a row	After a turn the car is disarmed. It needs 20 readings in a row that are not its color before it can turn again
It crosses the other color's line (orange on an anticlockwise run)	Ignored, and it counts toward re-arming
No pulse comes back before the 5 ms timeout	That channel reads 0, so the reading becomes OTHER instead of a false line
A false line reading while armed	Not handled: the car would turn early and the count would be off by one
A line is missed completely	Not handled: the count stays one short and the car keeps going straight
Clockwise run (right turns)	Steering at 109° is a mirrored estimate and still needs full test runs

The three orange rows are the known gaps in v2. All of them come from counting lines instead of knowing where the car actually is. Our newest Open Challenge version also reads the ultrasonic sensors.

3.3 Loop speed and motor control

The faster the car reads the floor, the less distance it covers between readings, and the closer to the line it starts its turn. So we cut the time one reading can take.

	Version 1	Version 2
Samples per decision	`3 channels × 3 = 9`	`3 channels × 1 = 3`
Wait after switching filter	`3 ms`	`0.3 ms`
Pulse timeout	`30 ms`	`5 ms`
Worst case per decision	**`9 × (3 + 30) = 297 ms`**	**`3 × (0.3 + 5) = 15.9 ms`**

That is about **19 times faster** in the worst case. The cost is that version 2 no longer averages, so a single noisy reading counts. The 20-reading re-arm rule (page 17) protects against that on the line itself.

Printing R, G and B to serial every loop also takes time. It is very useful for tuning, and it is the first thing we would cut if we need more speed.

Driving the motor through the DRV8871

IN1	IN2	Motor
0	0	Coast (off)
1	0	One direction
0	1	Other direction (forward on our wiring)
1	1	Brake

DRV8871 input logic, from the TI datasheet.

Our `driveForward()` holds IN2 HIGH and sends PWM to IN1 as `255 - speed`. So the motor switches between driving forward and braking, and spends speed/255 of the time driving.

`brakeMotor()` sets both pins HIGH for 300 ms to stop hard at the finish, then lets the motor coast. That stops the car closer to where we want than just cutting power.

3.4 Obstacle Challenge at the qualifier

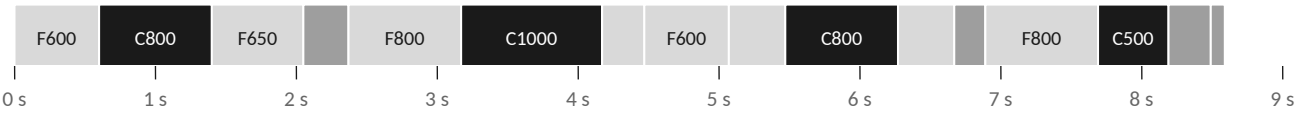
For the qualifier we ran the Obstacle Challenge **on timing alone**. The car followed a fixed list of moves (straight runs, small corrections around the obstacles, harder left turns at the corners), each for a set number of milliseconds. No camera, no ultrasonics, no color sensor.

It was the fastest way to a car that finishes, and it helped get us to the final. But it only works for the one layout it was tuned on, and it drifts as the battery runs down. **The obstacle program for the final is being rebuilt around the HuskyLens 2** (page 20).

Lap sequence 1, runLap() 18 moves, 9090 ms



Lap sequence 2, runLap2() 16 moves, 8590 ms



Both timed sequences on the same time scale. F forward, L and R small corrections, C corner (dark), numbers in ms.

Move	Servo	PWM	Use
`FORWARD(ms)`	82°	150	Straight
`LEFT(ms)`	65°	150	Small left correction
`RIGHT(ms)`	92°	150	Small right correction
`CORNER_LEFT(ms)`	55°	220	Corner, then back to 82°

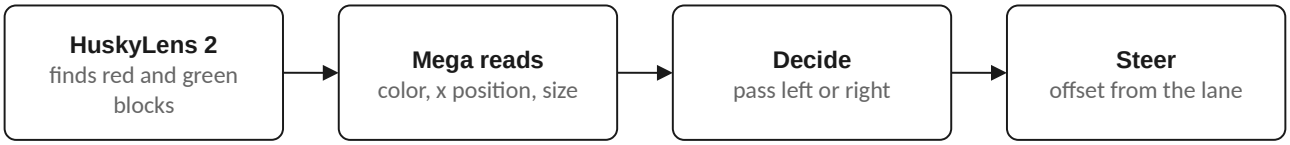
Total: 300 ms start pause + 9090 + 8590 ms of moves = about 18 s from button release to stop, plus a little execution time.

Note: this code was written for the version 1 wiring (L298N on D5, D32 and D33, servo on D7). It is kept in Appendix B as a record, and won't run on the current wiring without changing the pins.

3.5 Obstacle Challenge with the HuskyLens 2

The rules we have to meet. Pass red obstacles on the right and green ones on the left (rule 9.19). Direction, start position and obstacle layout are all drawn at random (chapter 8), so a fixed route can't work. After three laps the car has to park parallel in the parking lot (worth up to 15 points), all within 3 minutes.

Status. The camera is mounted and wired to UART1, and the power problem is solved with the power bank. Next is reading detections on the Mega.



Planned obstacle pipeline. Corners still come from the color sensor, exactly like the Open Challenge.

Plan, in order

Step	Goal	Done when
1 Teach the colors	Reliable red and green detection under field lighting	Both colors detected reliably anywhere on the mat
2 Read on the Mega	HuskyLens results over UART1 with the DFRobot HUSKYLENS 2 library	Color and x position print over serial while driving
3 Steer from them	Pass red on the right, green on the left, hold center when nothing is seen	Clean passes on single obstacles
4 Full laps	Reuse the Open Challenge corner logic for both directions	Three laps on several random layouts
5 Parking	Find the lot and park parallel	Parked inside the lot

Edge cases we already know about

Detection rate while moving. Two 2026 teams dropped the original HuskyLens because it updated too slowly at speed and their steering oscillated. We will measure how often fresh results arrive while driving before we trust it.

Nothing in view. The car needs a safe default. Our plan is to hold the lane center until a block shows up.

Two obstacles in one section. Our plan is to act on the closest one, judged by its size in the image.

4.1 Constraints and tradeoffs

What we had to work within

Rules	300 × 200 × 300 mm, 1.5 kg, 3-minute rounds, direction and layout drawn at random
Controller	One Arduino Mega: fine for sensors and motors, too small to process camera images itself
Platform	RN AI kit chassis, steering and rear drive
Time	Not enough before the qualifier to get vision working

We chose X instead of Y

We chose	Instead of	Because	What it costs us
Rear differential	Solid rear axle	Wheels don't skid in corners, turns repeat better	More parts and some play in the gears
Ackermann servo steering	Steering by driving the wheels at different speeds	Car-like turns with one servo and one motor	Bigger turning circle
Downward color sensor for lines	Using the camera for lines too	Simple, fast, works with the Mega directly	Only sees what is right under it
Timed turns	Gyro or encoder-based turns	No extra sensors, quick to tune	Turns change as the battery drains
Direction from the first line seen	Two programs, one per direction	One program, no chance of loading the wrong one	A false first reading would set the wrong direction
DRV8871 driver	L298N (version 1)	MOSFET driver: smaller and loses less voltage than the L298N's transistors	Rewiring and new motor code
HuskyLens 2 on UART	Raspberry Pi + camera	Detection runs on the camera, so the Mega stays the only controller	Less control over the vision code. Few teams to learn from
4S LiPo + 12 V step-down	12 V battery straight into Vin	Motor and Mega get a steady 12 V as the LiPo drains from 16.8 V, so timed moves stay consistent	Everything depends on one regulator. LiPo needs careful charging
Separate power bank for the camera	Camera on the Mega's 5 V	The Mega's regulator can't supply 0.3 to 0.6 A from 12 V	A second battery and switch to remember

4.2 Risks and failure modes

Two of these already happened to us. The rest are things we expect could go wrong at the final, and what we do about them.

What can fail	Seen?	Effect on a run	What we do about it
Cable caught in a wheel	happened	Cut wire, dead sensor	Zip ties, clearance check before every run
Camera not getting enough power	happened	Camera never boots	Own USB power bank, + wire left disconnected
Power bank switches itself off	risk	Camera goes dark mid-run	10-minute idle test, check it is on before each run
Missed or false line reading	risk	Corner count off, car stops in the wrong place	20-reading re-arm rule, recalibrate at the venue
Different lighting at the venue	risk	Color ranges stop matching	Sensor LEDs, re-log readings before the first run
LiPo drained too far	risk	Step-down can't hold 12 V, cells get damaged	Check pack voltage, recharge before about 13.2 V
Servo spikes on the Mega's 5 V	risk	Mega resets mid-turn	Watch for resets, give the servo its own 5 V if it happens
All 12 V loads through one step-down	risk	Voltage drops or module overheats under motor load	Check the module's current rating against the 3.6 A motor peak
Clockwise run with untested 109°	risk	Wrong turn radius on right turns	Full clockwise test runs before the final
Too wide at inspection	risk	Not allowed to run	Measure width with the power bank fitted
Forgetting a switch	risk	Car or camera off at the start	Pre-run checklist with both switches

Pre-run checklist

- [ ] Main switch on, power bank on, HuskyLens screen showing
- [ ] Steering centered at 82° after power-up
- [ ] Turn the wheels by hand: nothing touches the wheels or linkage
- [ ] Battery charged
- [ ] Correct sketch uploaded for this challenge
- [ ] Car placed, button pressed and released

4.3 Testing

How we test. One change at a time, then a run on the mat with the Serial Monitor open. Every logged run gets a number, the code version, the direction and what happened, so we can tell which change actually helped.

Open Challenge, version 1 (before the qualifier)

Run	Direction	Goal	Result
01	anticlockwise	Tune steering	Used to tune the steering
02	anticlockwise	Tune steering	Used to tune the steering
03	anticlockwise	Full course	**3 laps, stopped correctly**
04	anticlockwise	Full course	**3 laps, stopped correctly**
05	anticlockwise	Full course	**3 laps, stopped correctly**

3 out of 3 full-course runs finished after tuning. We didn't time them or count wall contacts, and version 1 could only drive anticlockwise.

Version 2 test matrix for the national final

Each cell gets filled in as we run it: laps completed, stopped in the finish section (yes or no), time and wall contacts.

Start position	Anticlockwise	Clockwise
Straight section 1		
Straight section 2		
Straight section 3		
Straight section 4		
Inner wall moved in		
Inner wall moved out		

What counts as a pass: three laps, stopped fully inside the finish section, no wall contact. Clockwise runs matter most, since the 109° right steering is still an estimate.

5.1 Build it and run it

Everything needed to rebuild the Invader from scratch, in order.

1 Build the chassis

Assemble the RN AI kit chassis, steering and rear drive with the differential, following the kit manual (pages 5 and 8 to 12).

2 Mount the electronics

Upper deck: Mega with the screw-terminal shield, DRV8871, start button, power switch. Color sensor under the front at 27.5 mm. HuskyLens 2 on the front mast, power bank low in the middle.

3 Wire it

Follow the pin map on page 13. Battery through the main switch to the DRV8871 and to the regulator feeding Vin. Power bank to the HuskyLens 2 USB-C. Join every ground, but leave the camera cable's + wire unconnected.

4 Check before power

Steering moves freely from lock to lock, no cable can reach a wheel, polarity checked with a multimeter.

5 Upload

Arduino IDE, board Arduino Mega 2560, the Servo library. Open open-challenge-v2.ino and upload.

6 Calibrate

Open the Serial Monitor at 115200 baud and check the blue and orange readings on your mat against the ranges on page 15.

7 Run

Place the car in a straight section, press and release the button. It waits 300 ms, kicks off, and picks its direction from the first line. Reset before the next run.

5.2 Repository and version history

`github.com/Elzee-Studios/WRO-2026-Future-Engineers`

Folder	What's in it
`src/open_challenge`	Open Challenge sketch
`src/obstacle_challenge`	Obstacle Challenge sketch
`docs`	One file per rubric criterion
`hardware`	Bill of materials
`engineering-journal`	This notebook
`vehicle-photos, team-photos`	Six vehicle views, team photos
`video`	Run video links

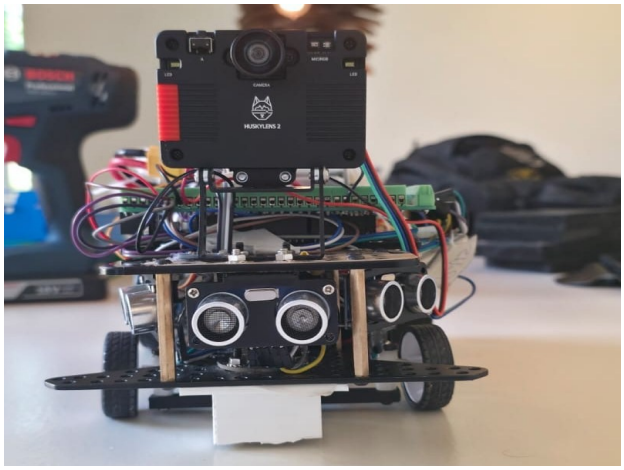
Version history

Version	When	What changed
v1	Qualifier	L298N driver, servo on D7. Open Challenge: left turns on blue only, 3-sample averaging, 1000 ms turns, 1250 ms finish. Obstacle Challenge: two timed lap sequences, no sensors.
v2	National final	DRV8871 driver on D5 and D6, servo on D8. Open Challenge: direction from the first line, separate first, normal and last turn times, kick start, brake at the finish, one sample per channel (worst case 297 ms down to 15.9 ms). HuskyLens 2 with its own power bank. 4S LiPo with a 12 V step-down for the Mega.
v2.1	National final	Open Challenge reads the ultrasonic sensors as well as the color sensor.
v3	In progress	HuskyLens 2 obstacle program, parking.

Keeping the repo honest. Each version above should be a tagged release, and each commit message should say what changed and why (for example “Move servo to D8, add clockwise turns”). The docs and bill of materials in the repo still describe v1 and need updating to match this notebook.

Photo record

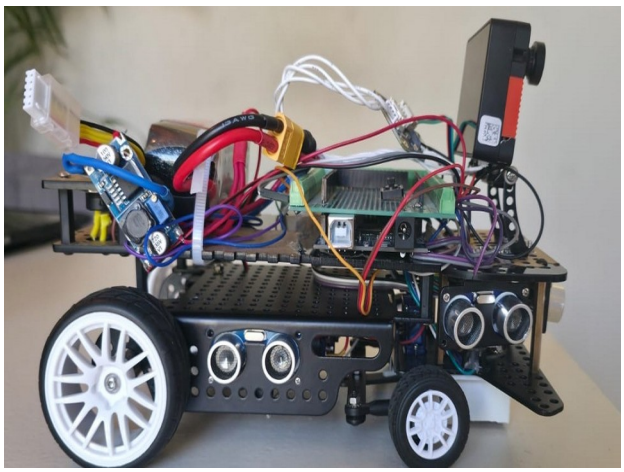
The six standard views of the car as it is now, with the DRV8871, HuskyLens 2 and 4S LiPo fitted.



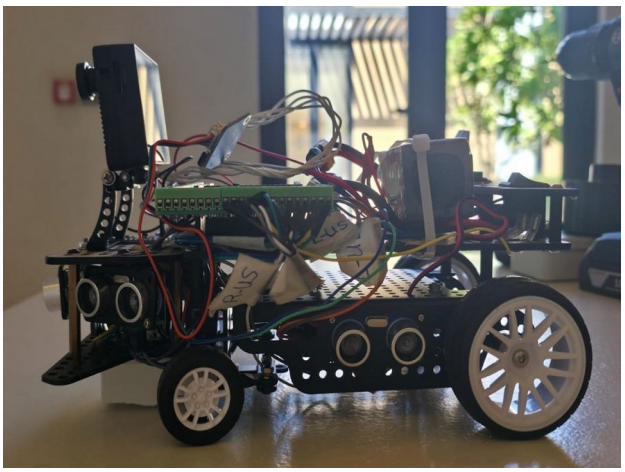
Front



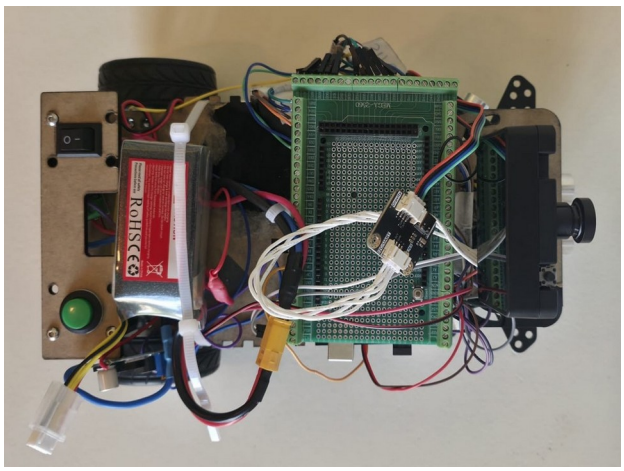
Rear



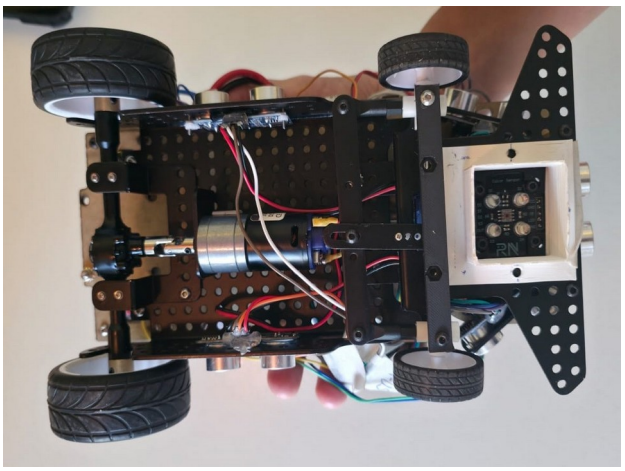
Right side



Left side



Top



Underside

Sources

1 **Our own build and test record**

Measurements, wiring repairs, calibration logs, test runs and photos.

2 **Our code**

open-challenge-v2.ino (Appendix A) and the qualifier obstacle-challenge-code.ino (Appendix B).

3 **RN AI kit user's manual**

Mechanical assembly, pages 5 and 8 to 12.

4 **WRO 2026 Future Engineers General Rules**

Rules 9.3, 9.19, 11.1, 11.2, 11.16 and Appendix C (documentation rubric). wro-association.org

5 **Texas Instruments DRV8871 datasheet**

Supply range, current limit and input logic.

6 **DFRobot HUSKYLENS 2 wiki (SEN0638)**

Supply voltage and power. wiki.dfrobot.com/sen0638

7 **Arduino Mega 2560 documentation**

Pinout, Vin range and power pins. docs.arduino.cc

8 **2025 International Final team repositories**

Double-X, KMIDS GFM, STORMS NGR, FireOnAllCylinders and ANTi, whose documentation we studied for structure. github.com/World-Robot-Olympiad-Association/fe-2025-links

A Open Challenge code, v2 (1/4)

`open-challenge-v2.ino` Blank lines left out to save space, line numbers match the file.

```
#include <Servo.h>
// DRV8871 motor driver
const int MOTOR_IN1 = 5;
const int MOTOR_IN2 = 6;
// Steering servo
const int SERVO_PIN = 8;
// Start button: D34 -> button -> GND
const int START_BUTTON = 34;
// TCS3200 color sensor
const int S0 = 35;
const int S1 = 36;
const int S2 = 37;
const int S3 = 38;
const int COLOR_OUT = 39;
// Movement settings
const int STEER_CENTER = 82;
const int STEER_LEFT = 55;
const int STEER_RIGHT = 109;  // Mirrored estimate: needs testing
const int NORMAL_SPEED = 200;
const int TURN_SPEED = 200;
const int KICK_SPEED = 255;
const unsigned long KICK_TIME_MS = 150;
const unsigned long FIRST_TURN_TIME_MS = 2200;
const unsigned long TURN_TIME_MS = 1800;
const unsigned long LAST_TURN_TIME_MS = 2100;
const unsigned long FINISH_DRIVE_MS = 1500;
Servo steering;
// 0 = not selected
// 1 = blue first: left turns, count blue only
// 2 = orange first: right turns, count orange only
int selectedColor = 0;
int cornerCount = 0;
bool lineArmed = true;
int nonSelectedReadings = 0;
// =========================================================
// MOTOR CONTROL
// =========================================================
void driveForward(int speedValue) {
  digitalWrite(MOTOR_IN2, HIGH);
  analogWrite(MOTOR_IN1, 255 - speedValue);
}
void stopMotor() {
  digitalWrite(MOTOR_IN1, LOW);
  digitalWrite(MOTOR_IN2, LOW);
}
void brakeMotor() {
  digitalWrite(MOTOR_IN1, HIGH);
  digitalWrite(MOTOR_IN2, HIGH);
  delay(300);
  stopMotor();
}
void kickStart() {
  driveForward(KICK_SPEED);
  delay(KICK_TIME_MS);
  driveForward(NORMAL_SPEED);
}
// =========================================================
// COLOR SENSOR
// =========================================================
unsigned long readColorChannel(bool s2, bool s3) {
```

A Open Challenge code, v2 (2/4)

`open-challenge-v2.ino` Blank lines left out to save space, line numbers match the file.

```
  79    digitalWrite(S2, s2);
  80    digitalWrite(S3, s3);
  82    delayMicroseconds(300);
  84    unsigned long pulse = pulseIn(COLOR_OUT, LOW, 5000);
  86    if (pulse == 0) {
  87      return 0;
  88    }
  90    return 500000UL / pulse;
  91  }
  93  // Returns: 0 = other, 1 = blue, 2 = orange
  94  int detectColor() {
  95    unsigned long R = readColorChannel(LOW, LOW);
  96    unsigned long G = readColorChannel(HIGH, HIGH);
  97    unsigned long B = readColorChannel(LOW, HIGH);
  99    Serial.print("R: ");
 100    Serial.print(R);
 101    Serial.print("   G: ");
 102    Serial.print(G);
 103    Serial.print("   B: ");
 104    Serial.print(B);
 105    Serial.print("   ---> ");
 107    // Blue range
 108    if (
 109        R >= 15000 && R <= 25000 &&
 110        G >= 15000 && G <= 24000 &&
 111        B >= 22000 && B <= 36000
 112       ) {
 113      Serial.println("BLUE");
 114      return 1;
 115    }
 117    // Orange range
 118    if (
 119        R >= 26500 && R <= 39000 &&
 120        G >= 19500 && G <= 25000 &&
 121        B >= 22500 && B <= 29000
 122       ) {
 123      Serial.println("ORANGE");
 124      return 2;
 125    }
 127    Serial.println("OTHER");
 128    return 0;
 129  }
 131  // ====================================================
 132  // TURN ACCORDING TO FIRST COLOR
 133  // ====================================================
 135  void turnForDirection() {
 136    if (selectedColor == 1) {
 137      Serial.println(">>> TURNING LEFT");
 138      steering.write(STEER_LEFT);
 139    } else {
 140      Serial.println(">>> TURNING RIGHT");
 141      steering.write(STEER_RIGHT);
 142    }
 144    driveForward(TURN_SPEED);
 146    if (cornerCount == 1) {
 147      delay(FIRST_TURN_TIME_MS);
 148    } else if (cornerCount == 12) {
 149      delay(LAST_TURN_TIME_MS);
 150    } else {
 151      delay(TURN_TIME_MS);
```

A Open Challenge code, v2 (3/4)

`open-challenge-v2.ino` Blank lines left out to save space, line numbers match the file.

```
152     }
154     steering.write(STEER_CENTER);
155     driveForward(NORMAL_SPEED);
157     Serial.println(">>> TURN COMPLETE");
158   }
160   // =======================================================
161   // FINISH
162   // =======================================================
164   void finishRun() {
165     Serial.println(">>> 12 CORNERS COUNTED");
166     Serial.println(">>> MOVING INTO FINISH AREA");
168     steering.write(STEER_CENTER);
169     driveForward(NORMAL_SPEED);
171     delay(FINISH_DRIVE_MS);
173     brakeMotor();
174     steering.write(STEER_CENTER);
176     Serial.println(">>> STOPPED");
178     while (true) {
179       steering.write(STEER_CENTER);
180       stopMotor();
181       delay(100);
182     }
183   }
185   // =======================================================
186   // SETUP
187   // =======================================================
189   void setup() {
190     Serial.begin(115200);
192     pinMode(MOTOR_IN1, OUTPUT);
193     pinMode(MOTOR_IN2, OUTPUT);
194     stopMotor();
196     steering.attach(SERVO_PIN);
197     steering.write(STEER_CENTER);
199     Serial.println("CENTERING STEERING TO 82 DEGREES");
201     delay(500);
203     pinMode(S0, OUTPUT);
204     pinMode(S1, OUTPUT);
205     pinMode(S2, OUTPUT);
206     pinMode(S3, OUTPUT);
207     pinMode(COLOR_OUT, INPUT);
209     // 20% output-frequency scaling
210     digitalWrite(S0, HIGH);
211     digitalWrite(S1, LOW);
213     pinMode(START_BUTTON, INPUT_PULLUP);
215     cornerCount = 0;
216     selectedColor = 0;
217     lineArmed = true;
218     nonSelectedReadings = 0;
220     Serial.println("READY - PRESS BUTTON");
222     while (digitalRead(START_BUTTON) == HIGH) {
223       steering.write(STEER_CENTER);
224     }
226     delay(100);
228     while (digitalRead(START_BUTTON) == LOW) {
229       steering.write(STEER_CENTER);
230     }
232     delay(300);
234     Serial.println("START");
235     Serial.println("CORNER 0 / 12");
237     steering.write(STEER_CENTER);
```

A Open Challenge code, v2 (4/4)

`open-challenge-v2.ino` Blank lines left out to save space, line numbers match the file.

```
238   kickStart();
239 }
241 // =======================================================
242 // MAIN LOOP
243 // =======================================================
245 void loop() {
246   int color = detectColor();
248   if (selectedColor == 0 && (color == 1 || color == 2)) {
249     selectedColor = color;
251     if (selectedColor == 1) {
252       Serial.println("BLUE FIRST: LEFT TURNS / COUNT BLUE ONLY");
253     } else {
254       Serial.println("ORANGE FIRST: RIGHT TURNS / COUNT ORANGE ONLY");
255     }
256   }
258   if (selectedColor != 0 && color == selectedColor) {
260     nonSelectedReadings = 0;
262     if (lineArmed) {
263       lineArmed = false;
265       cornerCount++;
267       Serial.print("CORNER: ");
268       Serial.print(cornerCount);
269       Serial.println(" / 12");
271       turnForDirection();
273       if (cornerCount >= 12) {
274         finishRun();
275       }
276     }
277   } else {
279     nonSelectedReadings++;
281     if (nonSelectedReadings >= 20) {
282       lineArmed = true;
283     }
285     steering.write(STEER_CENTER);
286     driveForward(NORMAL_SPEED);
287   }
288 }
```

B Obstacle code, qualifier v1 (1/2)

`obstacle-challenge-code.ino` v1 wiring (L298N, servo on D7). Comments and blank lines left out.

```
#include <Servo.h>
Servo steering;
const int SERVO_PIN = 7;
const int MOTOR_PWM = 5;
const int MOTOR_IN1 = 32;
const int MOTOR_IN2 = 33;
const int START_BUTTON = 34;
void FORWARD(unsigned long ms) {
  steering.write(82);
  digitalWrite(MOTOR_IN1, HIGH);
  digitalWrite(MOTOR_IN2, LOW);
  analogWrite(MOTOR_PWM, 150);
  delay(ms);
}
void LEFT(unsigned long ms) {
  steering.write(65);
  digitalWrite(MOTOR_IN1, HIGH);
  digitalWrite(MOTOR_IN2, LOW);
  analogWrite(MOTOR_PWM, 150);
  delay(ms);
}
void RIGHT(unsigned long ms) {
  steering.write(92);
  digitalWrite(MOTOR_IN1, HIGH);
  digitalWrite(MOTOR_IN2, LOW);
  analogWrite(MOTOR_PWM, 150);
  delay(ms);
}
void CORNER_LEFT(unsigned long ms) {
  steering.write(55);
  digitalWrite(MOTOR_IN1, HIGH);
  digitalWrite(MOTOR_IN2, LOW);
  analogWrite(MOTOR_PWM, 220);
  delay(ms);
  steering.write(82);
}
void STOP() {
  analogWrite(MOTOR_PWM, 0);
  digitalWrite(MOTOR_IN1, LOW);
  digitalWrite(MOTOR_IN2, LOW);
  steering.write(82);
}
void runLap() {
  FORWARD(1000);
  CORNER_LEFT(800);
  FORWARD(350);
  RIGHT(320);
  FORWARD(900);
  CORNER_LEFT(800);
  FORWARD(300);
  LEFT(250);
  FORWARD(350);
  RIGHT(500);
  FORWARD(400);
  CORNER_LEFT(800);
  FORWARD(400);
  LEFT(220);
  FORWARD(800);
  CORNER_LEFT(500);
  RIGHT(300);
```

B Obstacle code, qualifier v1 (2/2)

`obstacle-challenge-code.ino` v1 wiring (L298N, servo on D7). Comments and blank lines left out.

```
258   LEFT(100);
262   STOP();
263 }
274 void runLap2() {
280   FORWARD(600);
282   CORNER_LEFT(800);
293   FORWARD(650);
296   RIGHT(320);
299   FORWARD(800);
302   CORNER_LEFT(1000);
309   FORWARD(300);
322   FORWARD(600);
329   FORWARD(400);
333   CORNER_LEFT(800);
340   FORWARD(400);
347   LEFT(220);
350   FORWARD(800);
354   CORNER_LEFT(500);
366   RIGHT(300);
368   LEFT(100);
371   STOP();
372 }
388 void setup() {
391   steering.attach(SERVO_PIN);
394   steering.write(82);
398   pinMode(MOTOR_PWM, OUTPUT);
399   pinMode(MOTOR_IN1, OUTPUT);
400   pinMode(MOTOR_IN2, OUTPUT);
404   pinMode(START_BUTTON, INPUT_PULLUP);
408   STOP();
419   while (digitalRead(START_BUTTON) == HIGH) {
421     steering.write(82);
422   }
425   delay(100);
432   while (digitalRead(START_BUTTON) == LOW) {
434     steering.write(82);
435   }
439   delay(300);
446   runLap();
448   runLap2();
449 }
460 void loop() {
462   STOP();
464   delay(100);
465 }
```